ICT ACADEMY OF KERALA

Project Report

PERSONAL EXPENSE TRACKER

MERN Stack Capstone Project

Submitted by: Anandhakrishnan C V

Institution: sahrdaya college of engineering and technology Thrissur

Course / Batch: B Tech CSE

Project guide: Dr.Anil Antony

Submission date: 24/10/2026

Report Sections

- Executive Summary
- Introduction
- Objectives
- Scope and Deliverables
- Methodology
- Project Activities
- Results and Findings
- Conclusion
- Appendix

Project URLs

Frontend: https://spendwise-expense-tracker-1h76.onrender.com

Backend API: https://spendwise-api-appu.onrender.com

GitHub: https://github.com/anandhakrishnan225719-dotcom/spendwise-expense-tracker

EXECUTIVE SUMMARY

The Personal Expense Tracker is a web application for recording and reviewing personal spending. It was developed as a MERN stack capstone using React for the user interface, Express and Node.js for the API, and MongoDB with Mongoose for persistent data. It addresses scattered expense notes by providing one place to record purchases, group them by category, and review monthly totals.

The application includes user registration and login, authenticated expense operations, category summaries, and search/filter controls. The backend validates requests and scopes expense queries to the authenticated user. The frontend and backend are hosted separately on Render, with data stored in MongoDB Atlas. The repository is on GitHub. The student confirmed the deployed application was working; no automated test suite is claimed in this report.

INTRODUCTION

Personal spending can be difficult to understand when purchases are recorded across receipts, messages, or memory. A digital tracker can make this information easier to enter and review. This project applies the MERN stack to build a full-stack system that stores expenses and presents summaries through a browser.

The application follows a client-server design. React collects user input and renders summaries. The Express API validates requests, verifies login tokens, and communicates with MongoDB through Mongoose. MongoDB Atlas provides the managed database service. This separates presentation, business rules, and persistence while keeping the scope manageable for a student capstone.

This report adapts the supplied internship report structure to a capstone project. It describes the implemented scope, development approach, application behavior, deployment, and remaining improvements.

OBJECTIVES

1. Build a responsive web interface for recording and reviewing personal expenses.
2. Provide registration and login so expense records can be associated with an owner.
3. Create REST API operations to add, list, edit, and delete expenses.
4. Persist user and expense data in MongoDB Atlas through Mongoose models.
5. Provide monthly totals, expense counts, category summaries, and search/filter support.
6. Deploy the frontend and backend and document setup and use.
7. Apply basic security practices: password hashing, token verification, input validation, and environment-based secrets.

SCOPE AND DELIVERABLES

Included scope

- Registration and login using email and password.
- Add, view, edit, and delete expenses for the signed-in account.
- Expense fields: name, amount, category, and date.
- Categories: Food, Transport, Shopping, Bills, Health, Entertainment, and Other.
- Monthly spending overview, expense count, top category, and category summary.
- Search/filter controls, loading/error/empty states, and responsive browser layout.
- Hosted frontend, hosted API, MongoDB Atlas persistence, and public GitHub repository.

Deliverables

Deliverable	Description

React client	Vite interface under personal-expense-tracker/client.
Express API	Node.js service under server, with auth and expense routes.
MongoDB models	User and Expense schemas; each expense has an owner reference.
Deployment	Frontend and API on separate Render services, connected to Atlas.
Source and documents	GitHub repository, this report, and setup/user documentation.

Out of scope

The current version does not include bank integrations, receipt scanning, recurring payments, budget alerts, CSV export, password recovery, email verification, or multi-currency conversion.

METHODOLOGY

Work followed an iterative build-and-check process. The scope was reduced to a practical minimum viable product: user accounts, expense CRUD, categories, and a monthly overview. The user interface and API were built separately and integrated through HTTP requests.

Layer	Technology	Responsibility
Frontend	React 19, Vite 8, JavaScript, CSS	Forms, account views, summaries, filters, API calls.
Backend	Node.js, Express 5	REST endpoints, validation, CORS, authentication, JSON responses.
Database	MongoDB Atlas, Mongoose 9	User and expense persistence, schemas, references, timestamps.
Security	bcryptjs, jsonwebtoken	Password hashing and signed tokens for protected endpoints.
Hosting	Render	Separate static frontend and backend web service.

Architecture and Data Flow

A signed-in user interacts with the React client. The client sends JSON requests to the API and includes a bearer token for protected operations. Express validates input and token middleware identifies the owner. Mongoose reads or writes only that owner's expense documents. The response returns to the client, which updates the overview and transaction list.

Entity	Fields	Purpose
User	name, email, password hash, timestamps	Stores account identity. Email is normalized and unique; password is a bcrypt hash.
Expense	name, amount, category, date, user reference, timestamps	Stores a purchase and links it to its account owner. Category is restricted to supported values.

API Contract

Method and path	Access	Purpose
GET /api/health	Public	Health response for deployment checks.
POST /api/auth/register	Public	Create account and return token/user details.
POST /api/auth/login	Public	Verify credentials and return token/user details.
GET /api/expenses	Authenticated	List current user expenses by date.
POST /api/expenses	Authenticated	Validate and save an expense.
PATCH /api/expenses/:id	Authenticated	Update an expense owned by the user.
DELETE /api/expenses/:id	Authenticated	Delete an expense owned by the user.

The API returns JSON and uses HTTP status codes to report success and errors. Validation checks the name length, positive amount, supported category, and date. Protected requests without a valid token are rejected.

PROJECT ACTIVITIES

1. Project planning

Defined the user goal and minimum screens/functions. Identified accounts, expenses, categories, and monthly summaries as core features.

2. Client setup

Created a React/Vite client and built the expense overview, form, transaction list, category summaries, and authentication screens. Centralized requests in the API service.

3. Server and database

Created the Express server, Mongoose User and Expense models, auth routes, and CRUD routes. Added server-side expense validation.

4. Authentication and ownership

Hashed passwords using bcryptjs and issued JWTs after registration or login. Middleware verifies tokens; expense operations are filtered by owner.

5. Integration and debugging

Connected client requests to the backend, configured CORS, checked API and MongoDB connectivity, and resolved environment, directory, and request issues.

6. Source control and deployment

Committed the project to Git, pushed it to GitHub, deployed the API and client on Render, and supplied configuration through environment variables.

RESULTS & FINDINGS

The project delivers a full-stack expense flow: users can create an account, sign in, record a purchase, and review an overview. Expense information is stored in MongoDB, and protected API operations are associated with the authenticated account. The student confirmed the deployed website was working.

Implemented features

Area	Implemented behavior
Accounts	Register/login; hashed passwords; JWT-protected expense routes.
Expense records	Create, list, edit, delete expenses with name, amount, category, date.
Overview	Monthly spend, expense count, top category, category-level spending.
Usability	Search/filter, input validation, loading/error/empty states.
Deployment	Client and API live on Render; API connects to MongoDB Atlas.

Verification and evidence

Development verification was manual: services were run, the API health response was checked, Atlas network access was configured, and the live application was used. The student reported successful operation after deployment. No automated unit, integration, or end-to-end test suite was run. Add dated screenshots of registration/login, an overview with expense data, and the hosted site if required for submission.

Security and operational observations

- Secrets are read from environment variables and should not be committed. The repository ignores .env files.
- Passwords are hashed; the API uses bearer tokens and checks record ownership.
- The client stores the login token in local storage. This is convenient but is more exposed if cross-site scripting is introduced; HttpOnly cookies could be considered for production.
- The current application has no documented automated test suite, rate limiting, password reset, or email verification.
- Render free services may take time to wake after inactivity; check service logs and configuration if requests fail.

CONCLUSION

The Personal Expense Tracker demonstrates how a React interface, Express API, and MongoDB database can work together to solve a focused personal-finance problem. It meets its core goals through account-based expense entry, ownership checks, CRUD operations, monthly summaries, a live deployment, and a GitHub source repository.

The project provided practical experience with API design, schema validation, authentication, environment configuration, debugging, version control, and cloud deployment. Recommended next steps are a repeatable test suite, pagination, data export, and budget alerts. Academic submission should include screenshots and verified access instructions while keeping secret values private.

APPENDIX

A. Live Links

Frontend: https://spendwise-expense-tracker-1h76.onrender.com

Backend health: https://spendwise-api-appu.onrender.com/api/health

GitHub: https://github.com/anandhakrishnan225719-dotcom/spendwise-expense-tracker

B. Project Structure

- spendwise/.gitignore
- spendwise/server/server.js
- spendwise/server/models/User.js and Expense.js
- spendwise/server/routes/auth.js
- spendwise/server/middleware/auth.js
- spendwise/personal-expense-tracker/client/src/App.jsx
- spendwise/personal-expense-tracker/client/src/services/api.js

C. Configuration Names

Service	Environment variable names
Backend	PORT, MONGODB_URI, JWT_SECRET, CLIENT_URL
Frontend	VITE_API_URL

Do not include live secret values in the report, README, screenshots, Git history, or public issue tracker. Use local .env files for development and private environment settings for hosting.

D. Submission Details

Student: [Student Name] | Institution: [Institution Name] | Course / Batch: [Course and Batch] | Guide: [Guide Name] | Date: [DD Month YYYY]

E. Reference Documents

MERN_Stack_Project_Guidelines_Student_Reference.pdf, supplied by the student; report structure adapted from report format.pdf, supplied by the student.